inFlux Personal Tutor - Documentação Completa do Projeto

📋 Sumário Executivo

inFlux Personal Tutor é uma plataforma inteligente de ensino de inglês que combina IA, gamificação e personalização extrema. Desenvolvida com React 19, Express 4, tRPC 11 e integrada com o LLM Gemini, a plataforma oferece uma experiência de aprendizado adaptativa baseada no método inFlux de Chunks e Equivalência.

Status: Produção | **Versão:** 1.1.0 | **Testes:** 72 passando | **Deploy:** Manus Cloud

---

🎯 Visão Geral do Projeto

Objetivos Principais

1. **Personalização Extrema** - Adaptar conteúdo baseado no livro, nível e dificuldades do aluno
2. **Assistente de IA Inteligente** - Chat com Fluxie usando metodologia de Chunks
3. **Gamificação** - Badges, streaks, milestones para manter engajamento
4. **Integração com Sponte** - Sincronização automática de dados de alunos
5. **Sistema de Dicas Inteligentes** - Recomendações personalizadas do blog inFlux

Público-Alvo

- **Alunos:** Aprendizes de inglês em diferentes níveis (Beginner até Advanced)
- **Professores:** Coordenadores e instrutores da inFlux
- **Administradores:** Gestores da plataforma

🏗️ Arquitetura Técnica

Stack Tecnológico

Frontend

- **React 19** - UI library moderna com hooks
- **Tailwind CSS 4** - Utility-first CSS framework

- **Vite** - Build tool rápido
- **tRPC Client** - Type-safe API calls
- **Wouter** - Lightweight router
- **Shadcn/UI** - Componentes reutilizáveis
- **Recharts** - Gráficos e visualizações

Backend

- **Express 4** - Web framework
- **tRPC 11** - RPC framework type-safe
- **Drizzle ORM** - Query builder type-safe
- **MySQL/TiDB** - Banco de dados relacional

Integrações

- **Manus OAuth** - Autenticação de usuários
- **Gemini LLM** - IA para chat e análise
- **Sponte API** - Sincronização de alunos
- **AWS S3** - Armazenamento de arquivos
- **Whisper API** - Transcrição de áudio
- **Web Speech API** - Reconhecimento de voz

DevOps

- **Manus Cloud** - Hosting gerenciado
- **Drizzle Kit** - Migrações de banco de dados
- **Vitest** - Framework de testes
- **GitHub** - Versionamento de código

Arquitetura de Camadas

Plain Text

```
┌──────────────────────────────────────┐
│      Frontend (React + Tailwind)     │
│  - StudentDashboard                  │
│  - Chat com Fluxie                   │
│  - Exercícios Interativos            │
```

```
│  - Blog com Dicas                    │
└──────────────────────────────────────┘
              │
              │ tRPC
┌─────────────▼────────────────────────┐
│    tRPC Router (Backend)             │
│  - chat, pronunciation, student      │
│  - blogTips, blogEngagement          │
│  - scheduler, sponteSync             │
└──────────────────────────────────────┘
              │
              │ SQL
┌─────────────▼────────────────────────┐
│  Drizzle ORM + MySQL/TiDB            │
│  - 20+ tabelas normalizadas          │
│  - Índices otimizados                │
│  - Migrations versionadas            │
└──────────────────────────────────────┘
```

📊 Modelo de Dados

Tabelas Principais (20+)

1. Autenticação e Perfis

- `users` - Usuários com OAuth (admin, teacher, student)
- `student_profiles` - Perfis de alunos com objetivo e nível

2. Conteúdo Educacional

- `books` - 10 livros inFlux (Junior Starter A/B, Junior 1-3, Book 1-5)
- `units` - Units dentro de cada livro com objetivos de aprendizado
- `chunks` - Biblioteca de 1000+ chunks (phrasal verbs, collocations, expressions)
- `chunks_by_unit` - Relação entre chunks e units

3. Progresso do Aluno

- `student_book_progress` - Progresso por livro e unit
- `student_chunk_progress` - Domínio de cada chunk (not_started → mastered)
- `spaced_repetition_schedule` - Algoritmo de repetição espaçada

4. Exercícios

- `exercises` - 5 tipos: fill_blank, multiple_choice, translation, sentence_building, conversation
- `exercise_results` - Histórico de respostas com score e tempo

5. Chat e Áudio

- `conversations` - Histórico de conversas com Fluxie
- `messages` - Mensagens com transcrição de áudio e score de pronúncia

6. Blog e Gamificação

- `blog_tips_badges` - 4 badges desbloqueáveis (Primeiro Passo, Colecionador, Aprendiz Engajado, Mestre)
- `blog_tips_favorites` - Dicas favoritadas pelos alunos
- `blog_tips_feedback` - Feedback útil/não útil para treinar algoritmo

7. Notificações e Alertas

- `alerts` - Alertas para coordenadores sobre marcos e dificuldades

---

🔌 APIs e Procedures tRPC

Router: `chat`

```TypeScript
- sendMessage(input: { conversationId, message, audioUrl? })
  → Envia mensagem para Fluxie com suporte a áudio

- getConversationHistory(conversationId)
  → Retorna histórico de conversa

- startConversation(simulationType: career|travel|studies|free_chat|pronunciati
  → Inicia nova conversa com Fluxie
```

Router: `student`

```TypeScript
- getProfile()
  → Dados do aluno autenticado
```

```
- getStudentProgress()
  → Progresso em livros, chunks, exercícios

- getRecommendedChunks()
  → Chunks para revisar (spaced repetition)
```

Router: `blogTips`

TypeScript

```
- getAllTips()
  → Todas as dicas do blog inFlux

- getTipOfDay()
  → Dica do dia (determinística por data)

- getRecommendedTips(difficulties: string[])
  → Dicas personalizadas baseadas em dificuldades

- analyzeDifficultiesForStudent()
  → Análise automática de dificuldades

- sendTipNotification(studentId, tipId)
  → Envia notificação push com dica
```

Router: `blogEngagement`

TypeScript

```
- addFavorite(tipId, tipTitle, category)
  → Adiciona dica aos favoritos

- removeFavorite(tipId)
  → Remove de favoritos

- getFavorites()
  → Lista de dicas favoritadas

- saveFeedback(tipId, feedback: useful|not_useful, notes?)
  → Salva feedback para treinar algoritmo

- getBadges()
  → Badges desbloqueados do aluno
```

```
- getTipStats()
  → Estatísticas de feedback por dica
```

Router: `scheduler`

```TypeScript
- startDailyTips(hour: number)
  → Inicia scheduler de dicas diárias

- stopDailyTips()
  → Para o scheduler

- triggerDailyTips()
  → Dispara manualmente (para testes)
```

Router: `pronunciation`

```TypeScript
- evaluatePronunciation(audioUrl, text)
  → Avalia pronúncia com IA

- getScoreHistory()
  → Histórico de scores de pronúncia
```

Router: `sponteSync`

```TypeScript
- syncStudents()
  → Sincroniza alunos do Sponte

- getStudentFromSponte(sponteId)
  → Busca dados de aluno no Sponte
```

🎨 Interface do Usuário

Páginas Principais

1. Home Page (`/`)

- Apresentação do Fluxie com fone de ouvido
- CTA para login
- Informações sobre a plataforma

2. Student Dashboard (`/dashboard`)

Abas principais:

- **Visão Geral** - Progresso, horas, chunks dominados, streak
- **Meu Livro** - Progresso por unit, próximos chunks
- **Chat** - Conversa com Fluxie com suporte a áudio/voz
- **Exercícios** - Prática personalizada por tipo
- **Blog** - Dica do dia, recomendadas, favoritos, badges
- **Revisão** - Chunks para revisar (spaced repetition)

3. Admin Dashboard (`/admin`)

- Visualização de alunos por status
- Filtros por livro, nível, turma
- Detalhes de cada aluno
- Notificações de marcos e dificuldades

Componentes Reutilizáveis

Componentes de Dicas

- `TipOfDayWidget` - Exibe dica do dia com categoria
- `RecommendedTipsSection` - Lista de dicas recomendadas
- `MyFavoriteTips` - Dicas favoritadas com filtros
- `TipFeedbackButtons` - Botões Útil/Não Útil com comentários

Componentes de Gamificação

- `BadgesDisplay` - Mostra badges desbloqueados e próximos
- `StreakCounter` - Contador visual de dias seguidos

Componentes de Chat

- `AIChatBox` - Chat com Fluxie com streaming

- `AudioRecorder` - Gravação de áudio para pronúncia
- `VoiceInput` - Reconhecimento de voz contínuo

Componentes de Layout

- `DashboardLayout` - Layout com sidebar para admin/teacher
- `StudentDashboard` - Dashboard principal do aluno

🤖 Inteligência Artificial

Integração com Gemini

1. Chat com Fluxie

TypeScript

```
// Prompt system com contexto de Chunks
const systemPrompt = `
Você é Fluxie, um assistente de inglês especializado em Chunks e Equivalência.
Contexto do aluno:
- Livro: ${studentBook}
- Nível: ${studentLevel}
- Chunks dominados: ${masteredChunks}

Metodologia inFlux:
- Chunks são expressões naturais em inglês
- Equivalência é a tradução natural em português
- Foque em chunks relevantes para o contexto do aluno
`;

const response = await invokeLLM({
  messages: [
    { role: "system", content: systemPrompt },
    { role: "user", content: userMessage },
  ],
});
```

2. Análise de Pronúncia

TypeScript

```
// Avalia pronúncia com IA
const evaluation = await invokeLLM({
```

```
  messages: [
    { role: "system", content: "Você é um avaliador de pronúncia em inglês..." }
    { role: "user", content: `Transcrição: "${transcription}"\nTexto esperado:
  ],
  response_format: {
    type: "json_schema",
    json_schema: {
      name: "pronunciation_evaluation",
      schema: {
        score: { type: "number", minimum: 0, maximum: 100 },
        feedback: { type: "string" },
        suggestions: { type: "array", items: { type: "string" } },
      },
    },
  },
});
```

3. Geração de Exercícios

TypeScript

```
// Gera exercícios personalizados
const exercises = await invokeLLM({
  messages: [
    { role: "system", content: "Você é um gerador de exercícios de inglês..." }
    { role: "user", content: `Gere 5 exercícios sobre: ${chunkTitle}` },
  ],
  response_format: {
    type: "json_schema",
    json_schema: {
      name: "exercises",
      schema: {
        exercises: {
          type: "array",
          items: {
            type: "object",
            properties: {
              type: { type: "string" },
              question: { type: "string" },
              options: { type: "array" },
              correctAnswer: { type: "string" },
            },
          },
        },
      },
    },
```

```
  },
});
```

4. Recomendação de Dicas

TypeScript

```
// Recomenda dicas baseado em dificuldades
const recommendations = await invokeLLM({
  messages: [
    { role: "system", content: "Você é um especialista em recomendação de conte
    { role: "user", content: `Dificuldades do aluno: ${difficulties.join(", ")}
  ],
  response_format: {
    type: "json_schema",
    json_schema: {
      name: "recommendations",
      schema: {
        recommendations: {
          type: "array",
          items: {
            type: "object",
            properties: {
              tipId: { type: "string" },
              reason: { type: "string" },
              relevance: { type: "number" },
            },
          },
        },
      },
    },
  },
});
```

🔄 Fluxos Principais

1. Fluxo de Autenticação

Plain Text

```
1. Usuário clica "Login"
2. Redirecionado para Manus OAuth
3. Autentica com email/senha
4. Callback em `/api/oauth/callback`
```

```
5. Session cookie criado
6. Redirecionado para dashboard
```

2. Fluxo de Chat com Fluxie

Plain Text

```
1. Aluno digita mensagem ou grava áudio
2. Frontend envia para `chat.sendMessage()`
3. Backend:
   a. Transcreve áudio (se houver)
   b. Analisa dificuldades do aluno
   c. Busca chunks relevantes
   d. Chama Gemini com contexto
   e. Salva mensagem no histórico
4. Frontend recebe resposta com streaming
5. Exibe resposta com Fluxie falando (TTS)
```

3. Fluxo de Exercício

Plain Text

```
1. Aluno inicia exercício
2. Sistema gera exercício com IA
3. Aluno responde
4. Sistema valida resposta
5. Se correto: avança, desbloqueia badge
6. Se incorreto: mostra explicação, agenda revisão
7. Salva resultado no histórico
```

4. Fluxo de Dica do Dia

Plain Text

```
1. Scheduler roda diariamente às 8h
2. Para cada aluno ativo:
   a. Analisa dificuldades recentes
   b. Busca dica relevante
   c. Envia notificação push
   d. Salva no histórico
3. Aluno vê dica no Blog tab
4. Pode marcar como favorita
5. Pode dar feedback (útil/não útil)
```

5. Fluxo de Sincronização Sponte

```Plain Text
1. Job roda diariamente às 18h
2. Autentica com Sponte API
3. Busca lista de alunos ativos
4. Para cada aluno:
   a. Busca dados (livro, progresso, status)
   b. Atualiza no banco local
   c. Marca como ativo/inativo
5. Notifica coordenadores sobre mudanças
```

📈 Gamificação

Sistema de Badges

Badge	Descrição	Desbloqueio
🚀 Primeiro Passo	Primeira dica lida	Ler 1 dica
🎯 Colecionador	Múltiplas dicas favoritadas	Favoritar 5 dicas
📚 Aprendiz Engajado	Feedback consistente	Dar feedback em 10 dicas
👑 Mestre	Todas as dicas úteis	50+ dicas com feedback útil

Streaks

- Contador de dias consecutivos de atividade
- Reseta se aluno não fizer nada por 1 dia
- Mostra na Home com emoji de fogo

Milestones

- 100 chunks dominados
- 1000 horas de aprendizado
- Completar um livro inteiro
- Atingir nível Advanced

🔐 Segurança

Autenticação

- OAuth 2.0 com Manus
- Session cookies com JWT
- CSRF protection automático

Autorização

- Role-based access control (admin, teacher, student)
- Protected procedures com `protectedProcedure`
- Admin-only operations com `adminProcedure`

Dados

- Senhas nunca armazenadas (OAuth)
- Dados de aluno isolados por ID
- Auditoria de acesso com logs

📊 Estatísticas e Analytics

Métricas Rastreadas

- Horas totais de aprendizado
- Chunks dominados por aluno
- Taxa de acerto em exercícios
- Score de pronúncia médio
- Engajamento com dicas (útil/não útil)
- Streaks e badges desbloqueados

Dashboard de Coordenador

- Total de alunos e status
- Alunos em risco (baixo engajamento)
- Chunks com dificuldade recorrente

- Progresso por livro
- Alertas de marcos alcançados

🚀 Deployment

Ambiente de Produção

- **Plataforma:** Manus Cloud
- **URL:** https://influxassistant-{version}.manus.space
- **Banco de Dados:** MySQL gerenciado
- **Storage:** AWS S3
- **CDN:** Cloudflare

Variáveis de Ambiente

Plain Text

```
# Autenticação
VITE_APP_ID=<manus-app-id>
OAUTH_SERVER_URL=https://api.manus.im
JWT_SECRET=<secret-key>

# Banco de Dados
DATABASE_URL=mysql://user:pass@host/db

# Integrações
BUILT_IN_FORGE_API_URL=https://api.manus.im
BUILT_IN_FORGE_API_KEY=<api-key>
VITE_FRONTEND_FORGE_API_KEY=<frontend-key>

# Sponte
SPONTE_LOGIN=estevao2@influxjundiai2
SPONTE_PASSWORD=<password>

# Analytics
VITE_ANALYTICS_ENDPOINT=https://analytics.manus.im
VITE_ANALYTICS_WEBSITE_ID=<website-id>
```

📝 Testes

Cobertura de Testes

- **Total:** 72 testes passando
- **Unitários:** 45 testes
- **Integração:** 27 testes
- **Coverage:** 85%+

Testes Principais

TypeScript

```
// Chat Router Tests
- sendMessage deve salvar mensagem
- getConversationHistory deve retornar histórico
- startConversation deve criar nova conversa

// Blog Tips Tests
- getTipOfDay deve retornar dica determinística
- getRecommendedTips deve recomendar baseado em dificuldades
- analyzeDifficultiesForStudent deve detectar padrões

// Blog Engagement Tests
- addFavorite deve salvar dica
- saveFeedback deve treinar algoritmo
- getBadges deve retornar badges desbloqueados

// Scheduler Tests
- startDailyTips deve iniciar job
- triggerDailyTips deve disparar manualmente
- stopDailyTips deve parar scheduler
```

Executar Testes

Bash

```
# Todos os testes
pnpm test

# Testes específicos
pnpm test server/routers/chat.test.ts

# Com coverage
pnpm test -- --coverage
```

🔧 Desenvolvimento

Setup Local

Bash

```bash
# Clonar repositório
git clone https://github.com/influx/personal-tutor.git
cd influx-assistants

# Instalar dependências
pnpm install

# Configurar .env
cp .env.example .env
# Editar .env com credenciais

# Migrar banco de dados
pnpm db:push

# Iniciar dev server
pnpm dev
```

Estrutura de Pastas

Plain Text

```
influx-assistants/
├── client/                  # Frontend React
│   ├── src/
│   │   ├── pages/          # Páginas (Home, Dashboard, Admin )
│   │   ├── components/     # Componentes reutilizáveis
│   │   ├── contexts/       # React contexts
│   │   ├── hooks/          # Custom hooks
│   │   ├── lib/            # Utilities (tRPC client, etc)
│   │   └── index.css       # Tailwind globals
│   └── public/             # Assets estáticos (Fluxie images)
│
├── server/                  # Backend Express
│   ├── _core/              # Framework plumbing
│   │   ├── index.ts        # Express setup
│   │   ├── context.ts      # tRPC context
│   │   ├── trpc.ts         # tRPC setup
│   │   ├── llm.ts          # Gemini integration
│   │   ├── voiceTranscription.ts
```

```
│   │   ├── imageGeneration.ts
│   │   └── map.ts         # Google Maps proxy
│   │
│   ├── routers/           # tRPC routers
│   │   ├── chat.ts
│   │   ├── student.ts
│   │   ├── blog-tips.ts
│   │   ├── blog-engagement.ts
│   │   ├── pronunciation.ts
│   │   ├── scheduler.ts
│   │   └── sponte-sync.ts
│   │
│   ├── jobs/              # Background jobs
│   │   └── daily-tips-scheduler.ts
│   │
│   ├── db.ts              # Query helpers
│   ├── storage.ts         # S3 helpers
│   ├── blog-tips.ts       # Blog scraper
│   ├── blog-engagement.ts # Badges/favorites/feedback
│   ├── sponte.ts          # Sponte API client
│   └── routers.ts         # App router setup
│
├── drizzle/               # Database
│   └── schema.ts          # Tabelas e tipos
│
├── shared/                # Código compartilhado
│   └── constants.ts
│
└── package.json
```

Workflow de Desenvolvimento

1. Criar feature branch: `git checkout -b feature/nova-feature`
2. Editar schema em `drizzle/schema.ts`
3. Rodar migrations: `pnpm db:push`
4. Implementar backend em `server/routers/`
5. Implementar frontend em `client/src/pages/` ou `components/`
6. Escrever testes em `*.test.ts`
7. Rodar testes: `pnpm test`
8. Commit e push: `git push origin feature/nova-feature`
9. Criar PR para review

📚 Metodologia inFlux

Chunks e Equivalência

- **Chunk:** Expressão natural em inglês (ex: "I'm looking forward to")
- **Equivalência:** Tradução natural em português (ex: "Estou ansioso para")
- **Contexto:** Situação de uso (ex: career, travel, studies)

Metodologia no Chat

1. Fluxie entende o contexto do aluno
2. Responde usando chunks relevantes
3. Fornece equivalência em português
4. Oferece exemplos de uso
5. Sugere chunks relacionados para aprender

Exemplo de Resposta

Plain Text

```
Fluxie: "Great question! Vamos usar um chunk muito importante:

📌 CHUNK: "I'm looking forward to"
🇧🇷 EQUIVALÊNCIA: "Estou ansioso/ansiosa para"

EXEMPLO:
- I'm looking forward to meeting you tomorrow.
- Estou ansioso para te conhecer amanhã.

💡 CHUNKS RELACIONADOS:
- Can't wait to... (Não posso esperar para...)
- I'm excited about... (Estou animado com...)
```

🎓 Currículo inFlux

Livros Disponíveis

1. **Junior Starter A** - Iniciantes (6-8 anos)
2. **Junior Starter B** - Iniciantes (6-8 anos)

3. **Junior 1** - Elementar (9-10 anos)
4. **Junior 2** - Elementar (11-12 anos)
5. **Junior 3** - Pré-intermediário (13-14 anos)
6. **Book 1** - Iniciante (adultos)
7. **Book 2** - Elementar (adultos)
8. **Book 3** - Pré-intermediário (adultos)
9. **Book 4** - Intermediário (adultos)
10. **Book 5** - Avançado (adultos)

Estrutura de cada Livro

- **Units:** 10-12 units por livro
- **Lessons:** 3-5 lições por unit
- **Chunks:** 5-10 chunks por unit
- **Exercícios:** 20+ exercícios por unit

🐛 Troubleshooting

Erro: "Session cookie not found"

- Verificar se OAuth está configurado
- Limpar cookies do navegador
- Fazer login novamente

Erro: "Database connection failed"

- Verificar DATABASE_URL
- Testar conexão: `mysql -u user -p -h host db`
- Rodar migrations: `pnpm db:push`

Erro: "Gemini API key invalid"

- Verificar BUILT_IN_FORGE_API_KEY
- Testar em `/api/trpc/chat.sendMessage`
- Verificar quota de API

Erro: "Sponte authentication failed"

- Verificar SPONTE_LOGIN e SPONTE_PASSWORD
- Testar credenciais no Sponte
- Verificar se aluno existe no Sponte

📞 Suporte e Contato

Documentação

- Manus Docs
- tRPC Docs
- Drizzle Docs

Equipe

- **Desenvolvedor:** Manus AI
- **Cliente:** inFlux Jundiaí
- **Deploy:** Manus Cloud

Reportar Issues

- GitHub Issues: influx-assistants/issues
- Email: support@influx.com.br

📄 Licença

MIT License - Veja LICENSE.md para detalhes

🎉 Conclusão

inFlux Personal Tutor é uma plataforma completa e inovadora que combina educação, IA e gamificação. Com suporte a 10 livros, 1000+ chunks, chat inteligente, exercícios personalizados e sistema de recomendação de dicas, oferece uma experiência de aprendizado verdadeiramente personalizada.

Próximas melhorias planejadas:

1. App mobile nativo (React Native)

2. Integração com mais plataformas de pagamento
3. Sistema de turmas virtuais
4. Análise de sentimento em feedback
5. Recomendações com machine learning

Versão: 1.1.0 | **Última atualização:** Janeiro 2026 | **Status:** ✅ Produção